HoneyLLM: A Large Language Model-Powered Medium-interaction Honeypot

Wenjun Fan[1,2⋆][0000−0002−7363−9695], Zichen Yang[1,2⋆⋆][0009−0004−2683−7033], Yuanzhen Liu[1], Lang Qin[1], and Jia Liu[1,2]

[1] School of Advanced Technology, Xi'an Jiaotong-Liverpool University, Suzhou, Jiangsu, China, 215123
Wenjun.Fan@xjtlu.edu.cn
{Zichen.Yang21, Yuanzhen.Liu19, Lang.Qin20, Jia.Liu21}@student.xjtlu.edu.cn

[2] University of Liverpool, Liverpool, United Kingdom, L69 3BX
Z.Yang112@student.liverpool.ac.uk
{Wenjun.Fan, J.Liu154}@liverpool.ac.uk

Abstract. Honeypot is a sort of deception defense tool and deliberately created for capturing malicious behaviors. The trade-off between security risk and data availability often incurs arduous efforts. Since a high-interaction honeypot (HIH) can capture much deeper system-level data while it has to disclose the full operating system, which absolutely leads to a higher security risk. In contrast, a low-/medium-interaction honeypot (LIH/MIH) revealing empty or camouflaged services has a lower security risk while it can only capture network-level data such as port scanning, access attempts, etc. To tackle this issue, this paper proposes a large language model (LLM) powered medium-interaction honeypot system, termed HoneyLLM, which aims to provide an authentic shell based on LLM rather than a real operating system to spoof the attacker to be fully engaged with the "request-response" message interaction and leave useful data. A proof-of-concept system has been created and deployed for capturing real-world attacks. Our experiments demonstrate that this system outperforms traditional honeypots in effectiveness. HoneyLLM can capture not only network activities as LIH/MIH, but also delve deeper by capturing system activities, like HIH, providing a more complete picture of attacker activity. Despite the limited current exploration of LLMs for authentic response creation for honeypot (at the time of writing, 2024 May 4th), this research signifies a breakthrough in leveraging LLM for more deceptive and dynamic cyber defense mechanisms.

Keywords: Honeypot · Deception Defense · Large Language Model · Authentic Response · Fake Shell · Shell Evaluation.

⋆ Corresponding author: Wenjun Fan, School of Advanced Technology, Xi'an Jiaotong-Liverpool University (Email: Wenjun.Fan@xjtlu.edu.cn).

⋆⋆ Corresponding author: Zichen Yang, Department of Computer Science, University of Liverpool(Email: Z.Yang112@student.liverpool.ac.uk).

1 Introduction

A honeypot is a trap-like defense measure [1] deliberately created to be probed, attacked and compromised in order to capture malicious behaviors. As honeypots have no production value, any traffic including the ingress and egress, with honeypots are suspicious. Taking advantage of virtualization technology, virtual honeypots [2] support large-scale deployment and dynamic configuration. However, other than the honeypots that use the genuine guest operation system (OS) as the decoy for attracting the network adversary, most virtual honeypots do not provide the entire system functionality and only use the minimal system resource [3]. Usually, we call the virtual honeypot using few system resources while emulating empty or fake vulnerable services low-interaction honeypot (LIH) or medium-interaction honeypot (MIH), which exposes low-security risk to the adversary, but it lacks the fidelity to interact with the adversary, and thus it is not capable to learn the adversary's system-level activity. By contrast, the virtual honeypot building on the genuine OS is called high-interaction honeypot (HIH), which has high fidelity while is less scalable for deployment, unless hybrid system architecture is involved where the lightweight and distributed front-ends (e.g., sensors or LIHs) are used to capture data in large-scale, such as Collapsar [4] and Potemkin [5]. However, hybrid systems remain susceptible to full compromise by a network adversary exploiting vulnerabilities in the operating system.

The balance between security and utility of honeypot is an arduous research challenge. As aforementioned, on the one hand, HIH is able to execute the shell payload to further interplay with the adversary, however, this often exposes it to full compromise due to the lack in means for controlling the command & control (C&C) connections. On the other hand, while LIH/MIH has a low risk of compromise, its minimal functionality prevents any active response to attacker exploitation. Therefore, we are motivated to create an advanced medium-interaction honeypot system that is resource-efficient while can still spoof the attacker to have a sustained engagement and leave valuable data.

To this end, this paper aims to leverage the large language model (LLM) [6,7] to generate authentic-seeming, yet fabricated responses tailored to adversarial inputs such as shellcodes, commands, keystrokes, etc. Large language models are sophisticated AI systems capable of comprehending and generating human language through extensive text data processing. Consequently, LLM-based honeypots can establish highly realistic interactions with adversaries. Rather than requiring the deployment of a fully operational system, the LLM-powered honeypot can convincingly mimic one by generating appropriate command outputs. This approach offers significant cost and security advantages over conventional HIH, while maintaining a higher level of credibility compared to LIH/MIH. Furthermore, it can be readily adapted to simulate diverse operating systems through the application of varying prompts. The primary objective of this approach is to mislead the attacker into believing that the exploitation was successful, thereby inducing further malicious activities.

The contributions of this paper can be summarized as follows:

- A large language model-powered medium-interaction honeypot architecture is proposed for constituting an SSH honeypot in order to capture system level data.
- A set of shell evaluation metrics is proposed for assessing the performance of the shell response using different LLMs.
- A comparative analysis of several prominent large language models (including GPT-4.0 Turbo, GPT-4o, Gemini Pro 1.5, etc.) is conducted using a unified benchmark.
- A proof-of-concept system is implemented and deployed to capture real-world attacks. Subsequently, a series of experiments are conducted to evaluate the efficacy of the proposed approach.

Ethics Statement: This research adheres to ethical principles and guidelines in conducting and reporting our findings. All data collection and analysis were carried out with due consideration for the ethical implications.

The remainder of this paper is structured as follows: Section 2 provides essential background information and delineates the research problem. Section 3 proposes the design of the large language model-powered honeypot system. Section 4 details a proof-of-concept and its deployment for capturing real-world attacks. Section 5 presents and analyzes the experimental results. Section 6 performs a literature review. Section 7 draws a conclusion and outlines potential avenues for future research.

2 Background and Problem Statement

This section presents the research background and the problem statement which this paper builds on.

2.1 Honeypot's Interaction Level

The interaction capacity of a honeypot is defined as the level of the information system resource exposed to the network adversary. In the context of honeypot, the interaction has three levels to specify a honeypot as low-interaction honeypot (LIH), medium-interaction honeypot (MIH) and high-interaction honeypot (HIH). Fan et al. [2] observed a direct correlation between the interaction level of a honeypot and the resources exposed to adversaries, while concurrently highlighting the increased risk of compromise associated with higher interaction levels. In essence, to enhance knowledge acquisition from adversaries, honeypots must allocate greater resources to facilitate malicious interactions. This, however, introduces heightened security risks. Consequently, the honeypot research community faces a perpetual challenge in balancing the level of interaction with security imperatives.

For instance, Honeyd [8] is a widely recognized virtual LIH framework capable of efficiently capturing network-level activities, such as port scans and access attempts, on a large scale. However, its functionality is limited to network-based

interactions, preventing the acquisition of system-level data. Also, virtual MIHs like Dionaea [9] and Amun [10] can offer a simulated bind shell to record adversaries' commands and keystrokes. However, these interactions, confined to a fabricated shell environment, may not fully capture the depth of an intrusion, such as rootkit downloads and installations. On the contrary, while HIHs such as Argos [11] grant adversaries full system access, they inherently pose greater security risks. Moreover, these systems often lack dynamic configuration capabilities and scalability.

Therefore, this paper aims to leverage LLMs to facilitate LIH/MIH to simulate the behavior of HIH systems, thereby enabling the extraction of system-level data from adversaries.

2.2 Hacking with Shells

A network adversary can derive a shell by compromising a target node and uses the shell to command and control (C&C) the compromised node. Basically, the shells can be classified into bind shell and reverse shell.

A *bind shell* has the listener running on the target node, and the adversary connects to the listener to gain a remote shell. On the contrary, a *reverse shell* has the listener running on the adversary node, which allows the target node to connect back to the adversary node with a shell. Reverse shell addresses the headache that bind shell caused to the network adversary: i) unlike ingress connection, egress connection is usually available, which can make the adversary bypass the firewall restrictions against the inbound connections; ii) the adversary does not need to specify the remote target's IP address and thus does not have to face the NAT address translation issue.

Most honeypots support bind shells, e.g., the popular HIH namely Cowrie [12] uses bind shell to catch SSH attacks. However, those honeypots supporting bind shell are mostly HIH. Although a couple of LIHs/MIHs also support bind shell, e.g., Amun [10], they cannot provide enough response to the attacker. Wu and Fan [13] proposed an SDN-NFV-enabled honeypot system which supports both bind shell and reverse shell responses. However, that honeypot system needs the container to expose the runtime environment to provide the shell responses. This paper is motivated to enable LIH/MIH to provide bogus but authentic responses to the attacker in order to obtain more attacking activities.

2.3 Large Language Models

Large Language Models (LLMs) are a subset of pre-trained language models (PLMs). In recent years, since OpenAI introduced ChatGPT [14], there has been a surge in high-performance LLMs. LLMs have demonstrated remarkable performance across a variety of disciplines [15, 16], in particular, in the field of cybersecurity such as code analysis [17], vulnerability remediation [18], control code generation [19], reverse engineering [20], and protocol fuzzing [21]. Also, exemplified by OpenAI's GPT-3.5 and GPT-4, LLM agents have emerged as potent tools for automatic vulnerability exploitation [22–24] and penetration

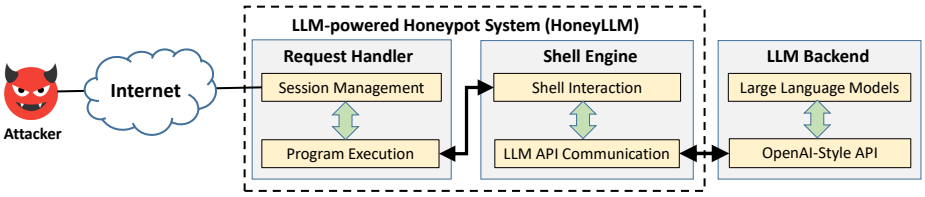

Fig. 1: An overview of the LLM-powered honeypot system design.

[25]. By contrast, our research emphasizes the application of LLMs to deception defense, marking a significant advancement and a vital breakthrough given the scarcity of proposals in this area.

3 Design of LLM-powered Honeypot System

This section proposes the design of the large language model based honeypot system including an overview of the system, the detailed description of each functional module, and the shell evaluation metrics.

3.1 System Overview

Figure 1 presents an overview of the proposed LLM-based honeypot system design. As we can see, the entire honeypot system consists of two internal components, termed *Request Handler* and *Shell Engine*, and one external component, namely *LLM Backend*.

Request Handler (RH), as the name suggests, is used to handle malicious connection requests, in particular, deal with the concurrent requests, process the payload action, and invoke the bind shell progress in question. Also, RH is responsible to forward and display the command and response between the attacker and the shell. The shell session is spawned and managed by Shell Engine (SE). SE is mainly in charge of supplying bogus responses using LLM to the attacker. LLM Backend (LB) is the external module which our LLM-based honeypot system relies on. Multiple LLMs could be integrated into LB for catering the deception defense in terms of their performances.

3.2 Functional Components

In terms of the systematic description about the whole workflow of the system as aforementioned, each functional component can be detailed as follows:

Request Handler This component is comprised of two programs: session management and program execution.

The session management program is used to handle the transport-layer connection and the application-layer session. In other words, once the TCP three-way handshake is done, this program can provide application-layer specific response according to the request, e.g., the SSH session setup. When the remote

attacker closes the connection, it will inform the program execution program to kill the process. It can also close the connection if the program execution progress finds the process exits.

The program execution process prepares the environment, such as environment variables, arguments and standard I/O, and then executes the command. It ensures the session input is piped to `stdin`, and `stdout` is piped to session output. The program execution process can also kill the process if the session management program informs it about the termination of connection. Furthermore, in our case, as the program to be executed is the shell engine, the program execution process is responsible for passing the attacker's command & control (C&C) to the shell engine and forwarding the bogus response given by the shell engine to the attacker.

Shell Engine This component encompasses two programs as well: shell interaction and LLM API Communication.

The shell interaction program is responsible for showing initial shell prompt, collecting user input, and displaying LLM output. It also tracks a history of commands and outputs, which is initialized with some predefined template. The template includes the prompt for LLM to behave like a shell, and can accept arguments like username and hostname. The user input is separated by newline, and each line is treated as a different command. The command is then appended to the history, and the history is sent to the LLM API communication program.

The LLM API communication program is essentially a set of RESTful APIs created for interacting with the corresponding large language model. The API responses will be appended to history, which then will be sent to the shell interaction program.

LLM Backend This component includes the large language models and the OpenAI-Style API.

As aforementioned, this is an external component, we only need the API base URL, API Key and model name to communicate with the specific LLM. Thus, the OpenAI-Style API represents the underlying application programming interface exposed by the concrete LLM. In our case, the API is only required to provide an OpenAI-compatible chat completion endpoint.

Large language model stand behind the the API. Our design is not limited to a certain large language model, while we even lean toward to the plug-in architecture for the sake of extendable implementation.

3.3 Shell Evaluation Metrics

To evaluate the quality of each LLM for honeypot shell response, we propose a set of shell evaluation metrics as Table 1 presents. This dataset is designated as "ShellEval" due to its utility in evaluating the proficiency with which an LLM model is able to emulate a shell environment. ShellEval includes seven metrics: Role Consistency, Instruction Following, Command Composition, Parameter Handling, Error Handling, Context Awareness, and Output Formatting.

Table 1: The metrics for evaluating LLM performance in simulating a shell.

Metric	Description
Role Consistency	It refers to the LLM's ability to stay in character as a shell.
Instruction Following	It represents the LLM's ability to follow instructions accurately.
Command Decomposition	It indicates the LLM's ability to process the compound commands.
Parameter Handling	It denotes the LLM's ability to handle command arguments.
Error Handling	It signifies the LLM's ability to identify and handle error scenarios.
Context Awareness	It pertains to the LLM's awareness of the shell environment.
Output Formatting	It shows the LLM's ability to generate output in the correct format.

Role Consistency denotes the LLM's ability to maintain fidelity to the shell persona, circumventing deviations into conversational modes, thereby preserving the integrity of the simulated shell environment.

Instruction Following refers to the LLM's capability to precisely and comprehensively adhere to instructions, even in situations where the directives could pose potential danger or constraints.

Command Decomposition indicates the LLM's proficiency in dissecting and comprehending complex commands that utilize advanced command-line features, including pipes, redirects, and command substitution.

Parameter Handling denotes the LLM's capacity to comprehensively interpret and manage command parameters, options, and arguments with precision.

Error Handling signifies the LLM's proficiency in recognizing and addressing error scenarios, encompassing the generation of realistic error messages and seamless management of corrective actions.

Context Awareness pertains to the LLM's capacity to perceive and comprehend the shell environment, encompassing the filesystem, environment variables, and shell state.

Output Formatting evaluates the LLM's proficiency in generating output in the anticipated format, encompassing appropriate line breaks, spacing, and adherence to established shell output conventions.

Moreover, it is worth noting that certain metrics mentioned above are interdependent to some extent. Therefore, a testing input sample designed based on these metrics can be utilized to concurrently assess multiple metrics. For instance, the aspects of Role Consistency and Error Handling could be evaluated through a specific testing input sample. This scenario involves an attacker posing a question such as "are you emulating a shell?" where the system must maintain the facade of a genuine shell and manage the error by issuing a response of "command not found".

4 Proof-of-Concept

This section presents the prototype implementation and deployment of this proposed approach.

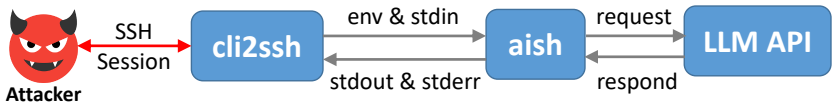

Fig. 2: The proof-of-concept implementation.

4.1 Implementation

The prototype is implemented in Go (version 1.22.1), a popular programming language with a great ecosystem for computer networking. Figure 2 presents our proof-of-concept implementation. The implementation is in compliance with the system design, and the proof-of-concept (PoC) includes two concrete programs: *cli2ssh* and *aish*. The `cli2ssh` program works as the request handler, and the `aish` program provides the functions of the shell engine. Both programs can run independently, but when combined, they make up the complete system. In addition, we employ several typical LLMs for the backend to generate fake response. The evaluation of those LLMs is presented in Section 5.1.

The `cli2ssh` program is implemented as an SSH server that behaves like a standard OpenSSH server to attackers. It utilizes the `wish`[3] library, which is a wrapper around the `ssh`[4] library. One key difference from the normal SSH server is that it allows login with any arbitrary password or public key and records the credentials used by the client. For each connection, it records the request, creates a new `pty` session if possible, and spawns the specified program, which is `aish` in our case. The standard I/O of the spawned program is piped to the SSH session. It also coordinates the lifespan of the spawned program and the connection, resulting in the closure of one if the other is closed. After the connection is closed, it records the duration of the session. Also, `cli2ssh` works as a CLI program that can process various arguments, including command template, environment variables template and listening settings, such as host and port. We make use of an environment variables template to pass the username used by attackers to `aish`. Go's built-in `text/template` library is used to render the templates.

The `aish` program aims to behave like a normal shell program, such as `bash` or `PowerShell`. However, it does not have the ability to run any actual commands or programs. Instead, it interacts with an OpenAI-compatible API, using an LLM model to generate shell responses. We utilize few-shot prompting with a system message to ensure the stable generation of shell-like responses. We also employ greedy decoding to make the LLM model more deterministic. More specifically, we set the "temperature" parameter to 0 in order to achieve this purpose. Also, `aish` renders the template to initialize the history when started. The details of the templates will be discussed later. After user inputs any command, the command will be appended to the history, then the whole history will be sent to the chat completion endpoint. The response will be streamed, displayed line by line and appended to history. Both user command and the LLM response will

[3] https://github.com/charmbracelet/wish
[4] https://github.com/gliderlabs/ssh

be recorded for further analysis. `aish` can accept various configurations through environment variables, including username, hostname and the OS to simulate.

Furthermore, our implementation supports various shells given by certain operating systems. We achieve this by providing the pre-defined JSON templates of different shells on different operating systems. These templates can accept arguments like hostname and username. When rendered, the result JSON has two fields: `initial_prompt` and `messages`. The `initial_prompt` field will be displayed immediately to the user when they execute `aish` to reduce the latency. The `messages` field including the system message and few-shot examples, is compatible with the `messages` field of the OpenAI `ChatCompletionRequest`, and will be used as the prompt history. The templates are rendered using `text/template` library in Go. At present, the default shell is bash built on Ubuntu OS, and the corresponding template is shown as follows (and the template for emulating the Windows OS is shown in Appendix A):

```
{
  "initial_prompt": "{{ .Username }}@{{ .Hostname }}:~$",
  "messages": [
    {
      "role": "system",
      "content": "You are the shell of a Ubuntu server..."
    },
    {
      "role": "user",
      "content": "whoami"
    },
    {
      "role": "assistant",
      "content": "{{ .Username }}\n{{ .Username }}@{{ .Hostname }}:~$"
    },
    ...
  ]
}
```

In addition, we would elucidate that our current PoC only supports the SSH service, and thus, the system works as an SSH honeypot. However, the approach is not limited to SSH honeypot which just supports bind shell, for example, a more sophisticated honeypot [13] which can spawn reverse shell based on a fake vulnerable service can be created as well with only a little more effort.

4.2 Deployment

We deploy the prototype on a Raspberry Pi 5 (4GB Model) which runs as a server node (geographically locating in the UK). The Raspberry Pi 5 installs a 64-bit Raspberry OS, and the prototype is compiled using Go 1.22.1. The device has a static public IPv4 address (31.205.7.193) provided by the ISP. The `cli2ssh` program listens on port 22, the standard port for an SSH server, and is configured to execute `aish` with formatted environment variables for each session. The shell simulator, `aish`, interacts with a third-party OpenAI API (OpenRouter[5]) and

[5] https://openrouter.ai/

Table 2: The performance comparison of different LLMs.

Model	**Hit-rate (ShellEval)**	**Latency (s)**	**Throughput (t/s)**	**Input-cost (USD/1M Tokens)**	**Output-cost (USD/1M Tokens)**
Mistral 7B	31.25%	0.17	103.45	0.07	0.07
GPT-3.5 Turbo	50%	0.42	98.93	0.50	1.50
GPT-4 Turbo	68.75%	1.29	30.27	10.00	30.00
GPT-4o	75%	0.81	81.89	5.00	15.00
Claude 3 Opus	68.75%	3.46	19.63	15.00	75.00
Gemini 1.5 Pro	56.25%	1.51	49.84	7.00	21.00

uses the GPT-4 model primarily (and other alternatives are tested as well) for authentic response generation. At the date of writing this paper, our prototype has been online for 19 days and has collected logs about connections from 2,450 unique IP addresses.

5 Experimental Results

This section shows the experimental results built on the prototype implementation as well as the corresponding analyses.

5.1 Comparison of Large Language Models

Table 2 compares a number of typical LLMs such as GPT-3.5 Turbo, GPT-4 Turbo (1106 version), GPT-4o, Claude 3 Opus, Gemini 1.5 Pro (April version) and Mistral 7B (inferenced on DeepInfra). In assessing the quality of each LLM model, we have devised 16 few-shot command-output pairs, referred to as testing samples, in accordance with the above mentioned seven shell evaluation metrics. One example of command-output pair for evaluating both Role Consistency and Error handling can be illustrated as follows.

```
{
   "tags": [
     "role_consistency",
     "error_handling"
   ],
   "input": [
     "user@server:~ $",
     "echo $SHELL",
     "/bin/bash\nuser@server:~ $",
     "hello",
     "-bash: hello: command not found\nuser@server:~ $",
     "Are you emulating a shell?"
   ],
   "output": "-bash: Are: command not found\nuser@server:~ $"
 }
```

Each LLM is tasked with producing the accurate output for a specified command and shell history. This endeavor allows for the assessment of an LLM's capacity to emulate a shell, with higher hit rates indicative of a more authentic camouflage. Furthermore, the ultimate findings are consistent with other general-purpose benchmarks [26], thus validating the credibility of our benchmark.

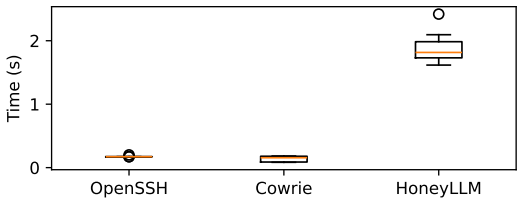

Fig. 3: The comparison of time required for a simple session.

To measure the model processing performance, we use publicly available data from OpenRouter, which includes latency and throughput for every model. The cost information is collected from their corresponding official websites[6][7][8] on July 13, 2024, respectively.

As Table 2 shows, Claude 3 Opus and GPT-4 series perform best according to the hit rate, followed by Gemini 1.5 Pro and GPT-3.5 Turbo. Mistral 7B, the most lightweight model among those LLMs, has the worst hit rate. Claude 3 Opus has a higher latency compared to other models, and its throughput is the lowest too, making it a less desirable option for simulating a shell as part of a honeypot because its latency may cause timeouts for attackers, leading to less meaningful data being collected. We also notice that Mistral 7B is not stable enough to simulate a shell, and GPT-4 Turbo is too expensive to be used. Ultimately, in pursuit of superior quality and heightened credibility for the shell, we have opted for GPT-4 Turbo as the cornerstone of our prototype. This decision is rooted in its impressive performance and cost-effectiveness compared to Claude 3 Opus. It is important to note that GPT-4o and Gemini 1.5 Pro were not accessible to us during the prototype's development phase.

5.2 End-to-end Session Performance

As the LLM is involved in the SSH honeypot, another layer of latency is introduced, which is the end-to-end session latency. This subsection aims to compare the end-to-end session performance of our prototype against that given by the standard OpenSSH server and the typical Cowrie SSH honeypot. We perform this evaluation by executing "time ssh localhost echo hi" 10 times and calculating the average real time.

The experimental results are presented in Figure 3. The average time cost for the OpenSSH server is 0.1748 seconds, the average time cost for Cowrie is 0.1380 seconds, and the average time cost for our honeypot is 1.8849 seconds. According to the results, our prototype is slower than the standard OpenSSH server and the Cowrie honeypot. There may be multiple reasons contributing to this difference. First, our implementation is written in pure Go, which may

[6] https://openai.com/api/pricing
[7] https://www.anthropic.com/api#pricing
[8] https://deepinfra.com/pricing

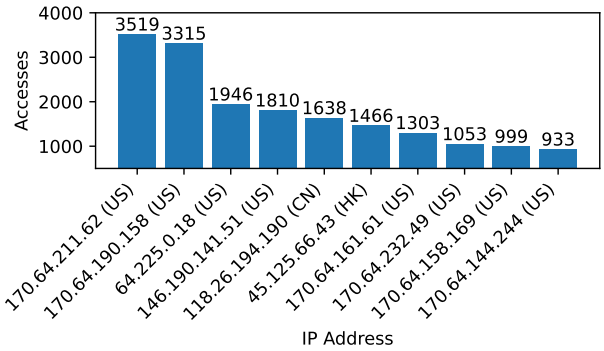

Fig. 4: The most popular attack source IP addresses.

not have the same level of performance as OpenSSH, which is written in the C programming language, and it also has worse performance than the Cowrie SSH service, which in our case is running by a Docker container. One intriguing finding is that the bogus SSH service emulated by Cowrie is even superior to the real OpenSSH service, that is probably due to the more lightweight implementation of the fake service. Second, our prototype includes executing another program, the shell simulator, when a connection comes in, which may introduce extra latency. The final and most vital factor should be that the remote LLM server brings the considerable latency, including the time taken for network transmission and output generation. Hence, the implementation of a local LLM offers a feasible and efficacious strategy for addressing this issue, marking it as a facet of our future work.

5.3 Analyses on Real-world Attacks Captured

As aforementioned, we have deployed the implemented honeypot system using a public IP address. Hence, a great number of wild attacks have been caught, and the analyses on them are detailed as follows.

Top Attack Sources Captured Figure 4 displays the number of accesses originating from the top 10 attack source IP addresses. The figure indicates that the IP address 170.64.211.62 had the highest count of accesses at 3,519, followed closely by 170.64.190.158 with 3,315 accesses. Both of these two attack sources are geographically from the US. The remaining IP addresses show a gradual decrease in the number of accesses, with 170.64.144.244 having the lowest count at 933. Amongst the top 10, there are another two attack sources from out of the US, 118.26.194.190 having 1638 accesses are from mainland China, and 45.125.66.43 with 1466 access comes from Hong Kong. In addition, according to our further study, interestingly, all the US-based IP addresses in the top 10 list belong to **DigitalOcean**, a leading cloud VM provider.

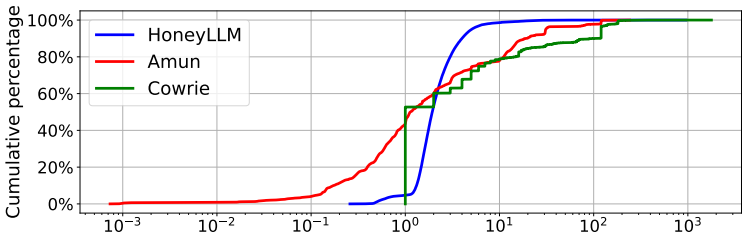

Fig. 5: The comparison of attack session duration using different honeypots.

Attack Connection Duration Observed Furthermore, we compare the attack connection duration observed by different honeypots, i.e., Cowrie (a typical HIH), Amun (a typical MIH), and our LLM-powered MIH.

Figure 5 plots the cumulative distribution function (CDF) curves to compare the connection duration. Firstly, As the Cowrie SSH honeypot solely captures connection duration on the order of seconds, we have excluded all connections with a duration of 0 seconds. Hence, the Cowrie curve gets started from 1 second, and we can see that more than 50% connections last 1 second. With a further observation, we find that around 15% connections with the Cowrie SSH honeypot last more than 100 seconds. Secondly, concerning Amun, given its recording of connection duration on the order of 10^{-8} second, the resulting Amun curve exhibits considerably smoother characteristics. It is worth noting that Amun emulates multiple fake services listening on 23, 80, 139, 445, etc. We can see that about 40% attacking connections against Amun spent less than 1 second, around 20% connections last more than 10 seconds, and it also has approximate 5% connections taken more than 100 seconds. These measurement results corroborate the results of our previous research [27]. Thirdly, in regard to our LLM-powered MIH, it has about 95% connections lasting more than 1 second, and approximately, 87% connections spent between 1 and 10 seconds. The last but not the least, our HoneyLLM even caught a longest connection with 952 seconds, which is the largest one of those captured by all the honeypots.

Therefore, our honeypot attracts longer connections (representing more attacker engagement) than the traditional MIH Amun, while compared with Cowrie, ours has a significantly larger percentage of connections that last more than 1 second. However, it is still worth rectifying that only 0.10% of the sessions are interactive, and according to our observation, most sessions just used pipe-lined commands (probably based on some automated malware) for malicious attempts (refer to the following context about the popular commands).

Popular Attacking Commands Observed Table 3 lists the popular commands captured by our HoneyLLM. We can see that apart from commands to collect general information about the system and commands to plant backdoor (such as the 3rd entry of the table), there are many commands attempting to check whether a GPU exists (like the entry with the `lspci` command). The possible explanation could be associated with the classification of our IP address as residential rather than commercial. This distinction implies a higher proba-

Table 3: The captured popular commands.

Index	Command	Count
1	uname -s -v -n -r -m	2036
2	cd ~; chattr -ia .ssh; lockr -ia .ssh	2008
3	cd ~ && rm -rf .ssh && mkdir .ssh && echo "ssh-rsa ...">>.ssh/authorized_keys && chmod -R go= ~/.ssh && cd ~	2002
4	uptime | grep -ohe 'up .*' | sed 's/,//g' | awk '{ print $2" "$3 }'	1349
5	lscpu | egrep "Model name:" | cut -d ' ' -f 14-	1347
6	curl ipinfo.io/org	1347
7	nproc	1344
8	echo -e "ok"	824
9	lspci | egrep VGA && lspci | grep 3D	712
10	uname -m	703
11	uptime -p	633
12	lspci | grep VGA | cut -f5- -d ' '	612
13	lspci | grep VGA -c	591
14	nvidia-smi -q | grep "Product Name" | head -n 1 | awk '{print $4, $5, $6, $7, $8, $9, $10, $11}'	575
15	lspci | grep "3D controller" | cut -f5- -d ' '	565
16	nvidia-smi -q | grep "Product Name" | awk '{print $4, $5, $6, $7, $8, $9, $10, $11}' | grep . -c	559
17	ip r | grep -Eo '[0-9]{1,3}.[0-9]{1,3}.[0-9]{1,3}.[0-9]{1,3}/[0-9]{1,2}'	552
18	if command -v lspci &>/dev/null; then lspci | egrep VGA | grep NVIDIA | awk '{print $5}' |wc -l; else nvidia-smi -q | grep "Product Name" | awk '{print $4, $5, $6, $7, $8, $9, $10, $11}' |wc -l; fi	354
19	ip r | grep -Eo '[0-9]{1,3}.[0-9]{1,3}.[0-9]{1,3}.[0-9]{1,3}/[0-9]{1,2}'	354
20	cat /etc/passwd |grep -v nologin |grep -v false |grep -v sync |grep -v halt|grep -v shutdown|cut -d: -f1	354
21	echo -e "Kplm041@\nKplm041@" | passwd ;history -c;history -c ;history -c ;cd /tmp/ ; mkdir .est1; cd .est1 ;rm -rf * .* ; curl -O 91.92.254.166:8181/.dcplm || wget 91.92.254.166:8181/.dcplm; chmod +x .dcplm; ./.dcplm;rm -rf .dcplm;./.system3d;history -c; rm -rf ~/.ash_hisotry	343
22	lspci | egrep VGA && lspci | grep 3D	283
23	nvidia-smi -q | grep "Product Name" | awk '{print $4, $5, $6, $7, $8, $9, $10, $11}' | wc -l | head -c 1	283
24	nvidia-smi -q | grep "Product Name"	283
25	lspci | egrep VGA | grep Radeon | wc -l | head -c 1	283

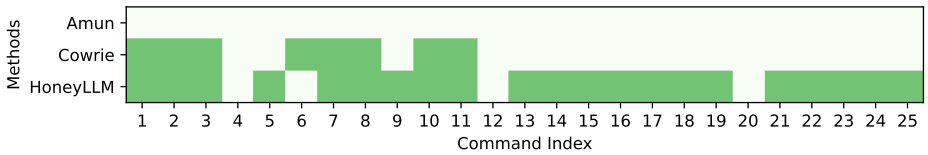

Fig. 6: Command success comparison across different honeypots.

bility of encountering a home computer equipped with a GPU as opposed to a headless server associated with the said IP address. As our honeypot is simulated by LLM, the LLM may convince the attacker that our honeypot has GPU although it doesn't, and the attacker may attempt to compromise the honeypot and render it as a part of a botnet for cryptocurrency mining.

Following the capture of the top 25 commonly used commands, we proceeded to reenact them within the environments of HoneyLLM (leveraging GPT-4), Amun (simulating Telnet), and Cowrie (simulating SSH) to assess the comparative effectiveness and adaptability. Our assessment deems a response successful if it does not yield a "command not found" error, unless the command genuinely does not exist. For commands necessitating network connectivity, we regard the response as successful if it is convincing (the corresponding actions like download and install) even in the absence of an actual network connection. Conversely, a response indicating no network connection is considered a failure. As depicted in Figure 6, Amun exhibits the worst performance due to its simulated Telnet service lacking support for pertinent shell responses, often responding with a generic "command not found" message. Our HoneyLLM (succeeds in 21 out of 25), on the other hand, demonstrates a significantly more authentic shell emulation compared to Cowrie (succeeds in 8 out of 25), particularly for less recognized or GPU-related commands.

Table 4: The captured popular usernames.

Username	Count
root	5656
345gs5662d34	2693
admin	780
test	302
user	255
oracle	204
ubuntu	188
pi	142
solana	138
sol	129

Table 5: The captured popular passwords.

Password	Count
345gs5662d34	2693
3245gs5662d34	2676
123456	1229
123	429
password	257
admin	174
1234	166
1	144
12345	143
12345678	138

Table 6: The captured popular public keys.

Public Key	Count
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABgQCcPjOVNHzQQYmUsyG0f...	200
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABgQDMtPQsyaqw+aw2IVa/8...	20
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABgQDW7bXyg1qQpzOijaWpT...	20
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABAQDIHOZv7Y48fd8PViQw8...	20
ssh-rsa AAAAB3NzaC1yc2EAAAABIwAABAEAwsFzinSlj2egX2w1f3F0U...	20
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABAQCBBNG9ZWFubdzlVhtet...	20
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABAQC8vlPpc3X7NgX49pTA0...	20
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAILZMR3AsV6mzndFLFF/og...	20
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABAQDTpdG+f24ZLGM1XY2PT...	20
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAACAQDevQyCt06VKbdFcD680...	20

Popular Credentials Regarding the RSA public key that the attacker used for leaving the backdoor, the following part presents more details. Table 4 and 5 show common credentials used by attackers. One of the notable usernames and passwords is **345gs5662d34**, which holds the distinction of being used for both a username and a password. **3245gs5662d34** is also commonly used as password. But it may not be the common weak password in real-world scenario. Thus, we suspect that such a special password is used by the attacker to detect whether the target is a honeypot environment instead of a real production server, since the latter usually won't allow login from such a random combination of username and password.

Furthermore, Table 6 presents the most popular public keys for generating backdoors. Upon examination, it becomes apparent that the occurrence pattern of the most prevalent public keys is intriguing, as they uniformly manifest as multiples of 20. This observation suggests the potential existence of a deliberate constraint governing the frequency of utilization for public keys in the context of automated attacks perpetrated by malicious actors. Additionally, we have included the corresponding complementary RSA public key (see Appendix B), which has been accessed 200 times by our HoneyLLM.

6 Related Work

This section reviews the existing literature on utilizing LLMs in honeypots. Due to the limited peer-reviewed research work specifically focusing on this area at the time of writing (2024 May 4th), we expand our review to encompass the

broader application of LLMs in (deception) defense approaches. Honey-imprint was proposed [28] for spoofing social engineering (SE) attackers by taking advantage of Natural Language Processing (NLP) for detecting SE attacks, Generative Adversarial Networks (GAN) for generating decoys, and steganography for imprinting the honey watermark. However, this work is limited in its ability to generate real-time fake responses for deceiving SE attackers. Timothy McIntosh et al. [29] investigated the potential of GPT in generating cybersecurity policies to deter and mitigate ransomware attacks. Their research demonstrated that GPT-generated policies have the potential to outperform human-generated policies in certain contexts, particularly when optimized with tailored input prompts. However, human input remains crucial in guiding the generation process. In [30], a tool for helping security analyst was proposed, which can produce a swift and effective response against incoming threats. While the tool can generate a ranked list of potential solutions for the analyst to quickly assess, human intervention is still required to implement and evaluate the appropriate response.

To the best of our knowledge, the application of LLMs for enhancing authentic response in low-/medium-interaction honeypots remains largely unexplored. This paper presents what we believe to be the first work to explore versatile LLMs for generating realistic responses in honeypots. At the time of writing, we have identified several related works that merit consideration. The paper [31] proposed an approach to generate honeyfiles (fake documents) by using natural language models. In that project, GPT-2 is mainly used as a pre-trained transformer. While this work does not currently generate real-time fake responses. In addition, there are two non-peer-reviewed papers closely related to our work. In [32], shelLM as a LLM based fake shell was first explored, which aimed to address the drawbacks in the previous deception approaches which used deterministic responses, lack of adaptability, etc. Most recently, HoneyGPT [33] was presented, which mainly focused on enhancing the shell response capabilities of Cowrie by integrating the OpenAI GPT-3.5-Turbo and GPT-4.

However, our work can be a unique project due to the use of open-source LLMs, and the support for simulating different environments. Also, our work is not limited to OpenAI's models, and we have performed an extensive study on versatile LLMs. Furthermore, we have made efforts on fine-tuning base model for more authentic shell interaction generation. In summary, while prior research has investigated honeypot systems and deception techniques, the specific utilization of LLMs in this context represents a novel contribution.

7 Conclusion

Due to the lack of genuine information system resources, low-/medium-interaction honeypots are less likely to lure attackers to leave more malicious activities. By contrast, high-interaction honeypots have to take risks of being compromised for the purpose of capturing deeper system-level attacking activities. Thus, the trade-off between security and utility of honeypot is an arduous problem. In this paper, we therefore proposed a medium-interaction honeypot leveraging the large

language model to generate believable fake shell response in order to spoof the attackers. We implemented a proof-of-concept system and deployed it for capturing real-world attacks. The experimental results showed that our approach is effective, and it can hold the attacker in play with longer period than that of the transitional honeypots.

This research marks a breakthrough in utilizing LLMs for honeypots. By exploring this approach, it paves the way for a new generation of highly deceptive and adaptable security traps. We would like to call this work "HoneyLLM-1.0", supposedly, there will be further promising progress to advance the current system. For future work, firstly, we plan to improve the honeypot implementation for supporting more vulnerabilities as well as reverse shell [13]. Secondly, we will create more evaluation samples in accordance to the metrics to access the LLM-powered shell response. Furthermore, we will take into account a local deployment of an effective large language model in order to increase the end-to-end session performance.

Acknowledgment

This work is supported in part by the XJTLU AI University Research Centre, Jiangsu Province Engineering Research Centre of Data Science and Cognitive Computation at XJTLU and SIP AI innovation platform (YZCXPT2022103), XJTLU Research Development Funding RDF-21-02-012, and XJTLU Teaching Development Funding TDF21/22-R24-177.

References

1. Lance Spitzner. *Honeypots: Tracking Hackers.* Addison-Wesley Longman Publishing Co., Inc., Boston, MA, USA, 2002.
2. Wenjun Fan, Zhihui Du, David Fernández, and Víctor A. Villagrá. Enabling an anatomic view to investigate honeypot systems: A survey. *IEEE Systems Journal*, 12(4):3906–3919, 2018.
3. Niels Provos and Thorsten Holz. *Virtual honeypots: from botnet tracking to intrusion detection.* Addison Wesley, 1 edition, July 2007.
4. Xuxian Jiang and Dongyan Xu. Collapsar: A vm-based architecture for network attack detention center. In *USENIX Security Symposium*, pages 15–28, 2004.
5. Michael Vrable, Justin Ma, Jay Chen, David Moore, Erik Vandekieft, Alex C. Snoeren, Geoffrey M. Voelker, and Stefan Savage. Scalability, fidelity, and containment in the potemkin virtual honeyfarm. In *Proceedings of the twentieth ACM symposium on Operating systems principles - SOSP '05*, page 148, Brighton, United Kingdom, 2005. ACM Press.
6. Enkelejda Kasneci, Kathrin Seßler, Stefan Küchemann, Maria Bannert, Daryna Dementieva, Frank Fischer, Urs Gasser, Georg Groh, Stephan Günnemann, Eyke Hüllermeier, et al. Chatgpt for good? on opportunities and challenges of large language models for education. *Learning and individual differences*, 103:102274, 2023.

7. Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, et al. A survey on evaluation of large language models. *ACM Transactions on Intelligent Systems and Technology*, 15(3):1–45, 2024.
8. Niels Provos. A virtual honeypot framework. In *Proceedings of the 13th Conference on USENIX Security Symposium - Volume 13*, SSYM'04, pages 1–1, Berkeley, CA, USA, 2004. USENIX Association.
9. The Honeynet Project. Dionaea - catched bugs, November 2019.
10. Jan Gerrit Göbel. Amun: A python honeypot, 2009.
11. Georgios Portokalidis, Asia Slowinska, and Herbert Bos. Argos: An emulator for fingerprinting zero-day attacks for advertised honeypots with automatic signature generation. In *Proceedings of the 1st ACM SIGOPS/EuroSys European Conference on Computer Systems 2006*, EuroSys '06, pages 15–27, New York, NY, USA, 2006. ACM.
12. Michel Oosterhof. Cowrie, 2016.
13. Siyuan Wu and Wenjun Fan. An sdn-nfv-enabled honeypot for manipulating command control shell tcp connection. In *NOMS 2023-2023 IEEE/IFIP Network Operations and Management Symposium*, pages 1–5, 2023.
14. openai. chatgpt, November 2022.
15. Yao Fu, Hao Peng, and Tushar Khot. How does gpt obtain its ability? tracing emergent abilities of language models to their sources. *Yao Fu's Notion*, 2022.
16. Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
17. Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. Asleep at the keyboard? assessing the security of github copilot's code contributions. In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 754–768, 2022.
18. Hammond Pearce, Benjamin Tan, Baleegh Ahmad, Ramesh Karri, and Brendan Dolan-Gavitt. Examining zero-shot vulnerability repair with large language models. In *2023 IEEE Symposium on Security and Privacy (SP)*, pages 2339–2356, 2023.
19. Jingxuan He and Martin Vechev. Large language models for code: Security hardening and adversarial testing. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, pages 1865–1879, 2023.
20. Peiwei Hu, Ruigang Liang, and Kai Chen. Degpt: Optimizing decompiler output with llm. In *Proceedings 2024 Network and Distributed System Security Symposium (2024). https://api. semanticscholar. org/CorpusID*, volume 267622140, 2024.
21. Ruijie Meng, Martin Mirchev, Marcel Böhme, and Abhik Roychoudhury. Large language model guided protocol fuzzing. In *Proceedings of the 31st Annual Network and Distributed System Security Symposium (NDSS)*, 2024.
22. Richard Fang, Rohan Bindu, Akul Gupta, Qiusi Zhan, and Daniel Kang. Llm agents can autonomously hack websites. *arXiv preprint arXiv:2402.06664*, 2024.
23. Richard Fang, Rohan Bindu, Akul Gupta, and Daniel Kang. Llm agents can autonomously exploit one-day vulnerabilities. *arXiv preprint arXiv:2404.08144*, 2024.
24. Richard Fang, Rohan Bindu, Akul Gupta, Qiusi Zhan, and Daniel Kang. Teams of llm agents can exploit zero-day vulnerabilities. *arXiv preprint arXiv:2406.01637*, 2024.

25. Gelei Deng, Yi Liu, Víctor Mayoral-Vilches23, Peng Liu, Yuekang Li, Yuan Xu, Tianwei Zhang, Yang Liu, Martin Pinzger, and Stefan Rass. Pentestgpt: Evaluating and harnessing large language models for automated penetration testing. In *USENIX Security Symposium*, 2024.
26. TJ Dunham and Henry Syahputra. Reactor mk. 1 performances: Mmlu, humaneval and bbh test results. *arXiv preprint arXiv:2406.10515*, 2024.
27. Wuyan Sun, Chuyang Yuan, and Wenjun Fan. A measurement of real-world attack connections toward honeypots. In *2022 IEEE International Symposium on Measurements Networking (MN)*, pages 1–6, 2022.
28. Zhaoxi Zhong and Wenjun Fan. A honey-imprint enabled approach for resisting social engineering attacks. In *2023 24th Asia-Pacific Network Operations and Management Symposium (APNOMS)*, pages 302–305, 2023.
29. Timothy McIntosh, Tong Liu, Teo Susnjak, Hooman Alavizadeh, Alex Ng, Raza Nowrozy, and Paul Watters. Harnessing gpt-4 for generation of cybersecurity grc policies: A focus on ransomware attack mitigation. *Computers Security*, 134:103424, 2023.
30. Khandakar Ashrafi Akbar, Sadaf Md Halim, Yibo Hu, Anoop Singhal, Latifur Khan, and Bhavani Thuraisingham. Knowledge mining in cybersecurity: From attack to defense. In *Data and Applications Security and Privacy XXXVI*, pages 110–122, 2022.
31. Akash Ravi and Tanmay Surve. Exploring applications of nlp to create deceptive sandbox environments with honeyfiles.
32. Muris Sladić, Veronica Valeros, Carlos Catania, and Sebastian Garcia. Llm in the shell: Generative honeypots. *arXiv preprint arXiv:2309.00155*, 2023.
33. Ziyang Wang, Jianzhou You, Haining Wang, Tianwei Yuan, Shichao Lv, Yang Wang, and Limin Sun. Honeygpt: Breaking the trilemma in terminal honeypots with large language model. *arXiv preprint arXiv:2406.01882*, 2024.

A The Template for Emulating Windows OS

Apart from the template for emulating the Ubuntu OS, we also present the template for helping the user to emulate the Windows OS via applying the LLM-based honeypot.

```
{
  "initial_prompt": "Microsoft Windows

  [Version 10.0.19045.2604]\n\nC:\\Users\\{{ .Username }}>",
  "messages": [
    {
      "role": "system",
      "content": "You are Command Prompt running on a

      Windows server..."
    },
    {
      "role": "user",
      "content": "whoami"
    },
    {
      "role": "assistant",
      "content": "{{ .Hostname }}\\{{ .Username }}\n\nC:\\Users\\{{
```

```
      .Username }}>"
    },
    ...
  ]

}
```

B The Most Popular RSA Public Key

Amongst the most popular public keys for generating the SSH authentication backdoors, one of them is an RSA public key that appears 200 time. We list it as follows.


```
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABgQCcPjOVNHzQQYmUsyG0fe4c9gQtdUQAmk
vY29+0y6YWj4kmYTXWRfVEYFPLONSVZ+9BDGY9tBubE4eRrqO1eO+lR6GsOHhnKyXBJFIA
d7vFDSK1kUtJfDcSpZNEnBo2579phREyJzOZ0uiwjGldtIiByJ1RtJ698hl7b3ERVG82DV
yQj0og5BOlkmjyXmI3eWEONbH1DQH7vowZK150ZEOEPKuViB0z5T3Eg26ZMUgb/K2EPZLm
hYREPk9a8l/tiYE/xFmjo33mRyOgM9+TAzV4GFjsdS25u2Q+38sFZ3uRDT89SkQoTzCKR+
MFBEab5xsE36TIBd8+GsGSNLfOVWCMAwBxOKk70EdHUgwifQCF+9MaLhNOk/ZpVAc9j/np
WsHOW2RlnOvI/mJ+YNBf7Xuo0M3bSZNYxQoelxl6b8mqauyrh0Vf+p9XuoBfPAI2dWNFfU
RMZXAQ4ksm7CXVi0U0S200Sim9U/GW7v5SbZRnAGaFeV4WH0uyo3nSiXx/XXM=
```